Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

SISTEMA DE ANÁLISIS EPIDEMIOLÓGICO MEDIANTE AUTÓMATAS CELULARES Y ESTRUCTURAS DE DATOS DINÁMICAS PARA LA DETERMINACIÓN DE GRAVEDAD CLÍNICA

202404224 – Alejandro Emmanuel Alvarez Velásquez

Resumen

El presente ensayo expone el desarrollo de un sistema de software diseñado para el laboratorio de investigación epidemiológica de Guatemala. La interacción entre células sanas y contagiadas determina patrones de comportamiento cíclico mediante reglas de supervivencia y contagio. A través de un motor de simulación iterativo, el sistema identifica si una patología resultará en un caso leve, grave o mortal. El enfoque técnico destaca la eficiencia en el manejo de memoria al almacenar únicamente las células con estado activo, garantizando el control total sobre la lógica de datos bajo restricciones de programación orientada a objetos. Los resultados demuestran que la detección de patrones periódicos es fundamental para el diagnóstico automatizado en contextos de alta complejidad algorítmica.

Palabras clave

Atómatas Celulares

Listas enlazadas

Epidemiologia computacional

Estucturas

Simulación

Abstract

This essay presents the development of a software system designe for the epidemiological research laboratory in Guatemala. It utilizes a model based on cellular automata on square grids of up to 10,000 cells per side. The interaction between healthy and infected cells determines cyclical behavior patterns through survival and contagion rules. Using an iterative simulation engine, the system identifies whether a pathology will result in a mild, severe, or fatal case. The technical approach emphasizes efficient memory management by storing only cells in an active state, thus ensuring complete control over the data logic under object-oriented programming constraints. The results demonstrate that the detection of periodic patterns is fundamental for automated diagnosis in contexts of high algorithmic complexity.

Keywords

Cellular Automata

Linked lists

Computational Epidemiology

Structures

Simulation

Introducción

La comprensión de la propagación de enfermedades a nivel celular es uno de los desafíos más críticos de la biotecnología moderna. En Guatemala, la necesidad de herramientas locales para el análisis predictivo ha llevado al desarrollo de modelos que simulan el comportamiento de tejidos bajo infección. Este ensayo detalla la construcción de un simulador que utiliza reglas de transición periódicas para evaluar la gravedad de una enfermedad. ¿Es posible determinar la mortalidad de un virus basándose únicamente en la repetición de patrones geométricos en un tejido? A través de una perspectiva inductiva, se analiza cómo la implementación de estructuras de datos dinámicas permite procesar volúmenes masivos de información celular (hasta 100 millones de posiciones), brindando un panorama claro sobre la trascendencia de la optimización algorítmica en la salud pública

1. Programación Orientada a Objetos

La programación Orientada a objetos se define como un paradigma de la programación, una manera de programar específica, donde se organiza el código en unidades denominadas clases, de las cuales se crean objetos que se relacionan entre sí para conseguir los objetivos de las aplicaciones.

Podemos entender la programación Orientada a objetos (POO) como una forma especial de programar, más cercana a como expresaríamos las cosas en la vida real que otros tipos de programación, que permite diseñar mejor las aplicaciones, llegando a mayores cotas de complejidad, sin que el código se vuelva inmanejable.

Al programar orientado a objetos tenemos que aprender a pensar cómo resolver los problemas de una manera distinta a como se realizaba anteriormente, en la programación estructurada. Ahora tendremos que escribir nuestros programas en términos de clases, objetos, propiedades, métodos y otras cosas que veremos rápidamente para aclarar conceptos y dar una pequeña base que permita soltarnos un poco con los conceptos de este tipo de programación.

2. Clase

Las clases son un concepto fundamental en la programación orientada a objetos. En términos simples, una clase es una plantilla que define la estructura y el comportamiento de un objeto.

Un objeto es una instancia de una clase. Por ejemplo, si tenemos una clase llamada "Coche", podemos crear varios objetos de esa clase, cada uno con sus propios valores para las propiedades de la clase, como la marca, el modelo y el color.

3. Abstracción

La abstracción en programación se refiere a la capacidad de representar y manejar conceptos complejos de manera simplificada. En otras palabras, es la capacidad de simplificar el mundo real y convertirlo en una «caja negra» que contiene solo la información esencial. Esta simplificación le permite a los programadores concentrarse en los detalles más

relevantes y abstractos de un sistema sin preocuparse por los detalles internos más complejos.

4. C#

Es un lenguaje de programación orientado a objetos desarrollado por Microsoft para crear aplicaciones para Windows. Hoy en día, C# se ha convertido en multiplataforma, pudiendo escribir programas tanto para Windows como para iOS y Linux, especialmente aplicaciones web y para el desarrollo de juegos.

El lenguaje se utiliza en el desarrollo del backend de aplicaciones web: en el trabajo con bases de datos y redes, en la programación de la lógica, etc. Los programadores de C# son solicitados en bancos, pequeñas empresas y nuevas empresas, y en el desarrollo móvil. C# es útil para aquellos que planean crear videojuegos y aplicaciones de RV en el popular motor Unity o trabajar con algoritmos de aprendizaje automático.

5. Grphviz

Graphviz constituye un software de código abierto de distribución gratuita orientado a la creación de representaciones gráficas, las cuales permiten visualizar datos estructurales en forma de diagramas. Su aplicabilidad es amplia y abarca campos tan diversos como el análisis de redes, la bioinformática, la ingeniería de software, las bases de datos, el diseño de páginas web y el aprendizaje automático. Además, proporciona interfaces gráficas diseñadas para diferentes dominios. (Takeyas, 2012)
El conjunto de herramientas Graphviz engloba una diversidad de programas diseñados para la creación de diagramas. Entre estas opciones, el lenguaje "dot" sobresale como la herramienta principal destinada a elaborar representaciones de estructuras jerárquicas o de capas dirigidas. Este lenguaje puede ser ejecutado como un programa desde la línea de comandos, a través de un servicio de visualización web o mediante una interfaz gráfica compatible. (Takeyas, 2012)

6. Tipos de Datos Abstractos

Es un modelo que define valores y las operaciones que se pueden realizan sobre ellos. Y se denomina abstracto ya que la intención es que quien lo utiliza, no necesita conocer los detalles de la representación interna o bien el cómo están implementadas las operaciones.
Es por esto una práctica que nos provee un grado de abstracción que permite desacoplar al código que usa un TDA de aquel código que lo implementa.

7. Archivos XML

XML consiste en un lenguaje de marcado creado por el W3C con la finalidad de definir una sintaxis para la codificación de documentos, que tanto los usuarios como las propias máquinas en sí puedan ser capaces de leer.
Para ello, lo hace mediante la utilización de una serie de etiquetas que definen la estructura que posee el documento en cuestión, además de cómo debe ser transportado y almacenado. (Pérez, 2020)
Podemos establecer una comparación con otro lenguaje de marcado con el cual es probable que estemos más familiarizados: el lenguaje de marcado de hipertexto (HTML), ampliamente empleado en la codificación de páginas web. HTML emplea una serie de símbolos de marcado predefinidos que delinean el formato del contenido de una página web. No obstante, una distinción fundamental entre XML y HTML radica en su extensibilidad. Mientras HTML

opera con un conjunto predeterminado de símbolos de marcado, XML se distingue por su naturaleza extensible.

Conclusiones

1. La implementación de estructuras de datos propias permite un control granular de la memoria, esencial para procesar mallas de alta densidad sin depender de librerías externas.

2. La simulación computacional permite transformar patrones visuales geométricos en diagnósticos clínicos precisos (mortal, grave o leve).

3. El uso de listas enlazadas para representar espacios bidimensionales (matrices dispersas) es la solución más eficiente para problemas donde la mayoría de los datos son nulos.

Referencias bibliográficas

C. J. Date, (1991). *An introduction to Database Systems.* Addison-Wesley Publishing Company, Inc.

https://digitalgrow.es/que-es-una-clase-en-programacion/

https://keepcoding.io/blog/que-es-la-abstraccion-en-programacion/

Deitel, P. J., & Deitel, H. M. (2017). C# 6 for Programmers. Prentice Hall.

Knuth, D. E. (1997). The Art of Computer Programming, Volume 1: Fundamental Algorithms. Addison-Wesley Professional.

Anexos

1. Tabla de actividades

Semana	Actividad
Primera Semana	Creacion del menu principal con algunas funciones.
Segunda Semana	Creacion de clases para el funcionamiento de el programa principal, como por ejemplo Celdas, listas celdas, etc.
Tercera Semana	Correccion de errores dentro del programa tomando en cuenta las restricciones, ademas la intalacion y el funcionamiento de Graphix.
Cuarta Semana	Verificar el funcionamiento de los xml, ademas la creacion de salida al momento de ejecutar las simulaciones.

Tabla 1. Cronograma de Actividades
Fuente: Elaboración Propia

2. Diagrama de Clases

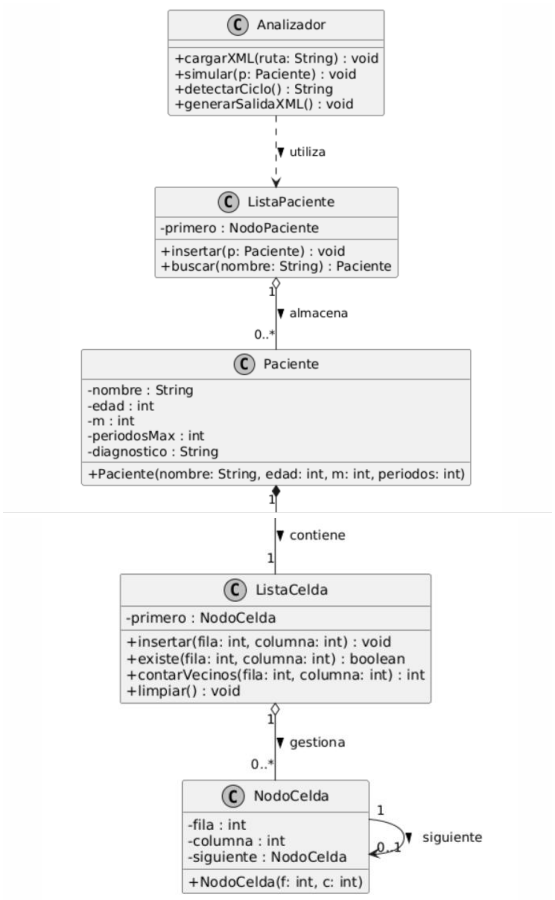

Figura 1. Diagrama de Clases
Fuente: Elaboración Propia

3. Diagrama de Listas

Figura 2. Diagrama de Listas
Fuente: Elaboración Propia